University of Southern Denmark

Bachelor Project

IMADA

Analysis of Real-World Developer Mistakes in configurable software systems

Student:
Salma Khair

Supervisor:
Dr. rer. nat. Sandra Greiner
Associate Professor

January 1, 2026

Contents

1 Introduction **1**
1.1 Introduction . . . 1

2 Background and State-Of-The-Art **3**
2.1 Variable Software Development . . . 3
2.1.1 Principles of Software Product Lines . . . 4
2.1.2 Classification of Edits to Configurable Software . . . 5
2.2 Principles of Natural Language Processing . . . 6
2.2.1 Basics . . . 6
2.2.2 Topic Modeling . . . 7

3 Implementation **8**
3.1 Preliminaries . . . 8
3.1.1 Supported Language Structure . . . 8
3.1.2 Tools and Libraries . . . 8
3.1.3 DiffDetective . . . 9
3.2 Architecture . . . 9
3.2.1 Overall Structure . . . 9
3.2.2 Specific Design Choices . . . 10
3.3 MyAnalysis . . . 10
3.3.1 Initialisation . . . 11
3.3.2 Classification . . . 11
3.4 Topic Analysis . . . 12
3.4.1 Input Data . . . 12
3.4.2 Keyword Frequency Analysis . . . 13
3.4.3 Output and Limitations . . . 13
3.5 Evaluation . . . 13
3.5.1 Setup . . . 13
3.5.2 Results . . . 13
3.5.3 Threats to Validity . . . 13

4 Discussion **14**

5 Conclusion **15**

Chapter 1 Introduction

1.1 Introduction

Modern software systems are rarely developed as a single, fixed program. Instead, many systems are **configurable**, meaning that the same code base can be compiled into different software variants depending on hardware, platform, or selected features. A common way to implement such configurability is through conditional code guarded by **presence conditions**, for example using preprocessor directives such as `#if` and `#ifdef`. Presence conditions determine in which configurations a particular code fragment is included and therefore directly influence the behaviour of the resulting software variants.

This bachelor thesis is situated in the area of **software engineering**, with a focus on configurable software systems and variability analysis. It studies how changes to conditional code, such as presence conditions (e.g. `#ifdef`), affect software systems over time by analysing large software repositories. In standard version control systems, changes are represented as textual diffs that show added or removed lines of code. While this representation is sufficient for many development tasks, it is often insufficient for configurable systems. In such systems, even a small textual change can have a large semantic impact if it modifies a presence condition.

For example, a minor edit to a conditional expression may silently expand or restrict the set of configurations in which a code fragment is active. These effects are not visible in standard diffs and are often undocumented in commit messages. As a result, developers and tools may overlook unintended changes to variability, which can lead to configuration-dependent faults that are difficult to detect.

The problem addressed in this thesis is therefore the limited ability of traditional diffs to explain the impact of changes to presence conditions. In particular, the thesis investigates how developer mistakes related to variability can be identified by analysing commits at the level of variability semantics rather than raw text differences.

To address this problem, this thesis builds on the **DiffDetective** tool to explicitly identify changes in variability. DiffDetective analyses software evolution by mapping source code and annotations to structured representations and by classifying edits according to their effect on variability. Instead of treating all changes as simple additions or deletions, DiffDetective distinguishes different kinds of variability edits, such as **refactoring** and **reconfiguration**. Based on this foundation, this thesis contributes a custom analysis tool called **MyAnalysis**. The purpose of MyAnalysis is to identify and store variability edits that are relevant for understanding developer mistakes.

The recorded results are later analysed using a topic-based approach, which groups refactoring and reconfiguration edits and collects their diff messages. This makes it easier to study variability-related diffs in more detail and to reason about recurring patterns in how such changes occur.

The analysis focuses mainly on two classes of edits: **refactoring** and **reconfiguration**. Refac-

toring describes changes where the structure of a presence condition is modified while its meaning remains the same. Reconfiguration describes changes where the artifact remains unchanged, but the set of configurations in which it is included changes in a non-trivial way. In addition, MyAnalysis extracts commit-level information such as commit identifiers and commit messages. A simple keyword-based heuristic is used to flag commits that are likely related to bug fixes. This allows the analysis to connect variability edits with developer intent as expressed in commit messages.

The solution is evaluated by applying MyAnalysis to a real-world configurable software system. By analysing how often variability-related diffs occur in practice, examining which types of diffs are most common, and studying the distribution of different variability edit types across large software repositories, the evaluation makes it possible to assess the overall extent of the problem.

This approach allows us to determine whether changes to presence conditions occur frequently and whether they represent a more substantial issue than initially expected. The results show that the analysis is able to distinguish between different types of variability changes and provides a more transparent view of how presence conditions evolve over time. By going beyond traditional textual diffs, the approach makes variability changes visible that would otherwise be difficult to recognise in standard version control output.

In the following chapters, this thesis is structured as follows. The next chapter provides background on configurable software and presence conditions and explains the motivation for using variability-aware differencing. The subsequent chapter describes the design and implementation of MyAnalysis. After that, the evaluation chapter presents the study and discusses the results. Finally, the thesis concludes with a summary of the main findings and an outline of possible directions for future work.

Chapter 2 Background and State-Of-The-Art

This chapter presents the theoretical background and state of the art relevant to this bachelor thesis. It introduces the fundamental concepts of **configurable software systems** and explains how variability is implemented using **presence conditions** and conditional compilation. Furthermore, it presents existing approaches for reasoning about edits to variable source code, including the classification of such edits. Finally, the chapter introduces basic principles of **Natural Language Processing (NLP)** that are later used to analyse commit messages. Together, these topics establish the conceptual basis for analysing developer mistakes related to presence conditions in configurable software systems.

2.1 Variable Software Development

Variable or configurable software systems are explicitly designed to support multiple variants within a single shared code base. As described by Apel et al., "a software product line is a set of software-intensive systems that share a common, managed set of features satisfying the specific needs of a particular market segment or mission and that are developed from a common set of core assets in a prescribed way" [**apel13**]. By selecting different configuration options or features, distinct software variants can be derived from the same underlying implementation. Each configuration determines which parts of the system are active and which are excluded, resulting in a concrete software product tailored to a specific context while maximising reuse of shared assets [**apel13**].

Such configurable software systems are widely used in practice, particularly in large and long-lived systems where maintaining separate forks would be infeasible. Operating systems are a prominent example: an operating system kernel must support different hardware architectures, device drivers, and optional subsystems, all of which are not needed in every deployment. Similarly, system libraries often provide optional functionality that can be enabled or disabled depending on performance requirements or supported platforms. Server applications frequently rely on configurability to support different protocols, security mechanisms, or deployment environments, while embedded systems must adapt to strict hardware constraints and diverse device capabilities. As noted by Apel et al., "variability is essential to adapt software systems to different execution environments, platforms, and requirements" [**apel13**]. Across these domains, configurability enables reuse of common functionality while still allowing systems to be customised to specific needs.

In real-world systems, variability is commonly implemented using directive-based mechanisms that conditionally include or exclude code. In languages such as C and C++, this is typically realised through conditional compilation using preprocessor directives such as `#if`, `#ifdef`, and `#ifndef`. These directives guard code fragments with logical expressions over configuration options. As observed by Liebig et al., "the C preprocessor is widely used to implement variability by conditionally including or excluding code fragments" [**DBLP:conf/icse/LiebigALKS10**]. Such expressions are referred to as presence conditions, as they specify under which configurations a particular code fragment is present in the compiled program. Given a concrete selection of configuration options,

all code whose presence conditions evaluate to false is excluded before compilation, resulting in a customised software variant [**apel13**].

While conditional compilation allows for variability at the level of individual statements or blocks of code, it also introduces substantial complexity. Presence conditions are often hierarchical and combined across multiple levels of the source code, leading to complex logical expressions that are not explicitly represented in a single location. Apel et al. emphasise that "conditional compilation easily leads to scattered and tangled variability, which complicates understanding and maintenance" [**apel13**]. As a consequence, the effective presence condition of a code fragment depends not only on its local annotation but also on all enclosing conditions. Liebig et al. similarly report that "understanding the impact of a change requires reasoning about the complete nesting structure of preprocessor directives" [**DBLP:conf/icse/LiebigALKS10**]. Understanding which configurations are affected by a given piece of code therefore requires reasoning about the complete conditional structure rather than an individual directive.

This complexity significantly affects how developers reason about variable software. Instead of dealing with a single program, developers must reason about a potentially enormous set of variants. In large systems, the number of valid configurations can grow into thousands or even millions, making exhaustive testing or manual inspection infeasible. As a result, developers typically rely on approximation strategies such as sample-based testing or family-based analysis rather than analysing all variants explicitly. As discussed by Apel et al., "due to the exponential number of variants, developers usually resort to sampling techniques or family-based analyses instead of analysing each variant individually" [**apel13**].

Despite the widespread use of configurable software systems, computer scientists and practitioners still face challenges in fully understanding and managing variability. Traditional software engineering tools, such as version control systems and diff tools, operate primarily on plain text and are largely unaware of the semantics of presence conditions. As noted by Apel et al., "existing development tools are mostly oblivious to variability and do not make the effects of variability explicit" [**apel13**]. As a result, changes to variability, especially changes to presence conditions, may not be immediately visible during code review. A seemingly small textual edit can unintentionally alter the set of configurations in which code is included, without this effect being explicitly documented or easily detectable.

This lack of tool support for configurable software increases the risk of developer mistakes when editing such systems. In particular, edits to presence conditions may introduce unintended changes to variability, potentially leading to configuration-dependent bugs that are difficult to detect and reproduce. Liebig et al. emphasise that "variability-related changes are hard to understand because their effects are often implicit and scattered across the code base" [**DBLP:conf/icse/LiebigALKS10**]. Understanding how variable software systems are implemented in practice, and how developers interact with presence conditions during evolution, is therefore a crucial prerequisite for analysing real-world developer mistakes in configurable software systems.

2.1.1 Principles of Software Product Lines

To reason systematically about variability in configurable software systems, the concept of a software product line provides an important foundation. Rather than focusing on individual variants in isolation, a software product line captures the idea that multiple related software products can be derived from a shared set of assets by varying certain characteristics in a controlled way. As defined by Apel et al., "a software product line is a set of software-intensive systems that share a common, managed set of features and are developed from a common set of core assets" [**apel13**].

A central concept in this context is that of a *feature*. A feature represents a user-visible char-

acteristic or capability of a software system. Features describe functionality at a level that is meaningful to users or stakeholders, such as support for a specific platform, optional functionality, or performance-related behaviour. According to Apel et al., "features are increments in program functionality that are relevant to some stakeholder and are used to distinguish between different products of a product line" [**apel13**]. Features can be mandatory, meaning that they are present in all products of the product line, or optional, meaning that they are only included in some products. In more complex cases, features may be organised into alternative or grouped structures, where only specific combinations are allowed.

A configuration is a concrete selection of features that defines a specific variant of the software system. Not all combinations of features are valid, as constraints may restrict how features can be combined. These constraints are commonly captured using feature models, which specify which feature combinations are allowed and which are forbidden. Apel et al. describe feature models as "a compact representation of all products of a product line in terms of features and their relationships" [**apel13**]. Each valid configuration corresponds to a concrete software product, that is, an individual variant of the system that includes exactly the functionality specified by the selected features.

While a configuration describes a single software product, a software product line encompasses the entire family of related products that can be derived from different valid configurations. All products in the product line share common functionality but differ with respect to selected features. As emphasised by Apel et al., "software product lines aim at systematic reuse by managing commonality and variability in a disciplined way" [**apel13**].

2.1.2 Classification of Edits to Configurable Software

Edits to configurable software do not only affect the implementation of source code but may also change the variability information that determines in which software variants the code is included. In systems that rely on presence conditions, even small edits can significantly alter the set of configurations in which a code fragment is active. Understanding such changes therefore requires reasoning about the semantic impact of edits on presence conditions rather than only their textual representation. As noted by Bittner et al., "small syntactic changes can have a large semantic impact on the variability of a system" [**Bittner2022**].

To address this challenge, Bittner et al. (2022) propose the first complete and unambiguous classification of edits to variability in source code. Their classification is based on a formal model of variability and explicitly captures how edits affect feature mappings and presence conditions over time. According to Bittner et al., "we present a complete and unambiguous classification of variability edits that precisely describes how presence conditions evolve" [**Bittner2022**]. This classification serves as the conceptual foundation for the edit analysis conducted in this thesis.

Figure 2.1 provides an overview of the edit classification proposed by Bittner et al. (2022). The figure illustrates how variability-related edits are structured into distinct categories based on their semantic effect on presence conditions and feature mappings.

At a high level, edits are categorised according to whether they add, remove, or modify variability. The classification distinguishes edits that introduce or delete code together with variability information from those that change the presence conditions of existing code without modifying the code itself.

The classification defines eight fundamental edit categories. Additive edits include **AddWithMapping**, which describes the addition of new code together with a new presence condition, and **AddToPC**, which captures additions of code that reuse an existing presence condition. Subtractive edits include **RemWithMapping** and **RemFromPC**, which describe different forms of code

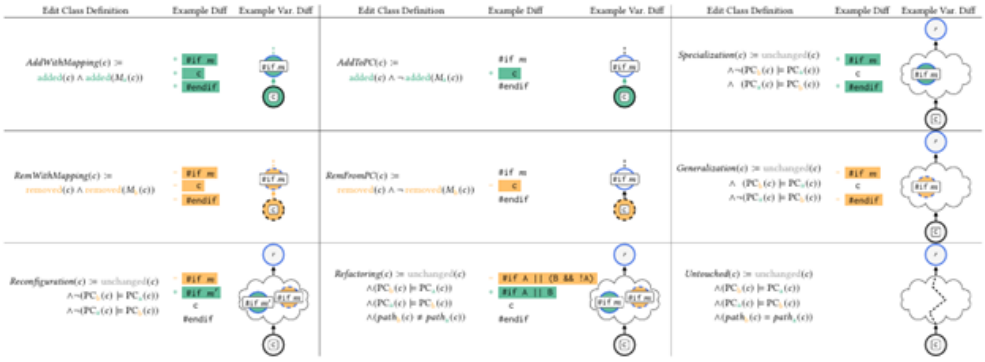

Figure 2.1: Classification of edits to configurable software proposed by Bittner et al. (2022).

removal with respect to variability annotations.

More subtle edits affect the variability of unchanged code. A **Generalization** makes a presence condition less restrictive, increasing the number of configurations in which code is included, while a **Specialization** makes a presence condition more restrictive. A **Reconfiguration** replaces a presence condition with a non-comparable alternative, leading to a non-monotonic change in the configuration space. Finally, **Refactoring** changes the syntactic structure of a presence condition while preserving its semantics.

A key contribution of this classification is that it is complete and unambiguous: every variability-related edit can be assigned to exactly one category. As stated by Bittner et al., "each variability edit belongs to exactly one edit class, making the classification complete and mutually exclusive" [**Bittner2022**]. Traditional line-based diff tools cannot capture these distinctions, as they ignore the semantics of presence conditions. The classification therefore provides a necessary abstraction for tools such as **DiffDetective** and for the analysis performed in this thesis.

2.2 Principles of Natural Language Processing

2.2.1 Basics

Natural Language Processing (NLP) is a subfield of computer science and artificial intelligence that focuses on the automatic analysis of human language. The central idea behind NLP is to transform unstructured text into representations that can be processed, analysed, and compared computationally. In software engineering research, NLP is commonly applied to textual artefacts such as commit messages, issue reports, and documentation.

While tools such as DiffDetective make variability-related edits explicit at the code level, commit messages remain a primary source for understanding why such edits were performed. NLP provides a systematic way to analyse these messages across large commit histories.

Despite its usefulness, NLP has inherent limitations, as it can only operate on the information explicitly provided in the text. Commit messages are intended to provide a complete summary of a change, but in practice they often cannot capture every technical detail, particularly when a single commit bundles multiple modifications. While commit messages are frequently informal in style, they may still omit information that is relevant for understanding the concrete impact of a change on the code base. Consequently, NLP techniques cannot compensate for missing or unreported

details and should therefore be used as a complementary technique alongside code-based analyses.

2.2.2 Topic Modeling

While DiffDetective identifies how presence conditions change at the source-code level, topic modelling is used to analyse commit messages in order to understand the contexts and intentions under which these changes were made. Topic modelling is an unsupervised NLP technique that discovers underlying thematic structures in a collection of documents.

One of the most widely used topic modelling techniques is **Latent Dirichlet Allocation (LDA)**, a probabilistic generative model that represents documents as mixtures of topics. In this thesis, topic modelling is applied to commit messages associated with edits to presence conditions identified by **DiffDetective** and `MyAnalysis`. Each commit message is treated as a document, and the resulting topics capture recurring themes in developer communication related to variability changes, such as bug fixing, cleanup of conditional compilation directives or refactoring and reconfiguration edits to presence conditions.

@articleblei2003lda, author = David M. Blei and Andrew Y. Ng and Michael I. Jordan, title = Latent Dirichlet Allocation, journal = Journal of Machine Learning Research, volume = 3, year = 2003, pages = 993–1022

Chapter 3 Implementation

This chapter describes the implementation of the analysis conducted in this bachelor thesis. It introduces the technical prerequisites of the implementation, outlines the supported language structures and tools, and presents the architecture of the analysis built on top of DiffDetective. The chapter further explains the specific design choices made for the custom analysis and discusses how the correctness of the implementation is ensured.

3.1 Preliminaries

This section describes the technical scope of the implementation and the tools used to realise the analysis.

3.1.1 Supported Language Structure

The implementation focuses on analysing configurable C/C++ software systems that use conditional compilation to express variability. In particular, the analysis supports variability implemented using preprocessor directives such as `#if`, `#ifdef`, `#ifndef`, `#else`, and `#endif`. These directives are used to guard code fragments with presence conditions that determine under which configurations the code is included.

The analysis does not operate on fully pre-processed code. Instead, it relies on annotated source code in which conditional compilation directives are still present. This focus is essential, as the analysis targets edits to presence conditions rather than fully configured variants.

Other variability mechanisms, including runtime configuration options and build-system-level variability, are outside the scope of this implementation. The analysis therefore concentrates on preprocessor-based variability, which is widely used in real-world C/C++ systems and well supported by existing variability analysis techniques.

3.1.2 Tools and Libraries

The implementation is realised using a combination of existing analysis frameworks and standard development tools. The analysis is implemented in Java and is designed to process real-world software repositories with publicly available version control histories.

Git is used to access and traverse commit histories, enabling the analysis of software evolution at the level of individual commits. More specifically, the JGit Java library is employed to programmatically analyse public GitHub repositories. Standard Java libraries are used for file handling,

data processing, and result serialisation. These libraries support reading source files, storing analysis results in a structured format, and preparing the output for further evaluation.

3.1.3 DiffDetective

DiffDetective is a variability-aware differencing Java library designed to analyse the evolution of configurable software systems. Unlike traditional line-based diff tools, DiffDetective explicitly models variability by representing conditional compilation directives and presence conditions as structured entities rather than plain text.

DiffDetective parses annotated source code and represents the association between code fragments and their corresponding presence conditions. When comparing two revisions of a source file, the framework detects differences in these associations, allowing variability-related edits to be identified even when the underlying code remains unchanged.

A central feature of DiffDetective is its ability to classify edits according to their semantic impact on variability. Based on the formal edit classification proposed by Bittner et al., DiffDetective distinguishes between different classes of variability-related edits, including refactoring, reconfiguration, generalisation, and specialisation. This classification provides a precise vocabulary for describing how presence conditions evolve across commits.

In this thesis, DiffDetective serves as the foundation for identifying and classifying edits to presence conditions. The framework provides the core analysis capabilities, while the custom analysis component builds on its results to extract relevant metadata and prepare the data for further evaluation and topic-based analysis of commit messages.

3.2 Architecture

3.2.1 Overall Structure

The execution of the analysis starts in the Java component and proceeds in a sequential manner. The entry point of the implementation is the class `Main.java`, which is responsible for configuring the analysis and connecting it to DiffDetective's execution infrastructure. `Main.java` defines the repository location and output directory and executes DiffDetective's traversal over individual commits. Importantly, this class does not implement analysis logic itself; instead, it delegates execution to DiffDetective, which ensures that the repository history is traversed consistently and reproducibly.

```
Path repoPath = Paths.get("repositories/postgresql");
Path outputDir = Paths.get("output");

Repository repo = Repository.fromDirectory(repoPath,
    repoPath.getFileName().toString());
Analysis.forEachCommit(() -> MyAnalysis.Create(repo, outputDir));
```

Listing 3.1: Entry point and configuration in `Main.java`

For each pair of successive commits, DiffDetective computes a variability-aware diff that captures changes to annotated source code, including the association between code fragments and their presence conditions. These diffs are represented internally using structured models rather than

plain text. The custom analysis, `MyAnalysis`, is executed within this traversal and receives access to both the current commit and the corresponding variability-aware diff.

Architecturally, this design cleanly separates control flow from analysis. Repository traversal, parsing of annotated source code, and computation of variability-aware diffs are handled entirely by DiffDetective. The custom implementation only observes and processes the results of these computations. This separation reduces complexity and ensures that the custom analysis remains focused on the specific goals of the bachelor thesis.

Lastly, a topic analysis is applied to structure the extracted results. This analysis groups related observations and supports the identification of recurring patterns in the evolution of variability across commits.

3.2.2 Specific Design Choices

A key characteristic of the implementation is that it builds a custom, commit-level analysis on top of an existing variability-aware differencing infrastructure rather than reimplementing repository traversal or differencing logic. By leveraging DiffDetective for commit traversal, parsing of annotated source code, and computation of variability-aware diffs, the implementation focuses exclusively on analysing and interpreting the resulting data.

Another specific aspect is that the analysis operates directly on real-world repositories with publicly available version control histories, without requiring preprocessing or manual annotations beyond those already present in the code base. The analysis is executed sequentially per commit, which enables fine-grained reasoning about how variability and presence conditions evolve over time.

Finally, the integration of a topic analysis step distinguishes the implementation by supporting higher-level interpretation of the extracted results. Rather than producing only raw diffs, the implementation structures observations into recurring themes, which makes it easier to evaluate how variability evolves over time.

3.3 MyAnalysis

MyAnalysis constitutes the custom analysis component of the implementation and is responsible for interpreting the variability-aware diffs computed by DiffDetective. Rather than performing repository traversal or syntactic differencing itself, MyAnalysis is integrated into DiffDetective's execution pipeline and operates exclusively on the intermediate results produced during commit traversal.

The analysis is implemented by conforming to the `Analysis.Hooks` interface provided by DiffDetective. This interface defines a set of lifecycle methods that are invoked at specific points during commit traversal. An instance of MyAnalysis is registered through a factory method, which constructs an `Analysis` object and attaches MyAnalysis as an analysis hook. Once registered, MyAnalysis is executed automatically for each commit processed by DiffDetective.

During execution, DiffDetective iterates over the commit history of the analysed repository. For each commit, it computes a variability-aware diff that captures changes to annotated source code together with their associated presence conditions. MyAnalysis accesses both the current commit object and the corresponding diff through the analysis context, allowing it to analyse commit metadata and variability-related changes in a unified manner.

3.3.1 Initialisation

```
@Override
public void initializeResults(Analysis analysis) {
    Files.createDirectories(outputDirectory);

    Path analysisFile = Paths.get("commit_analysis_log.txt");
    Path messageFile  = Paths.get("commit_messages.txt");

    Files.deleteIfExists(analysisFile);
    Files.deleteIfExists(messageFile);

    analysisWriter = new BufferedWriter(new
        FileWriter(analysisFile.toFile(), true));
    messageWriter  = new BufferedWriter(new
        FileWriter(messageFile.toFile(), true));

    writtenCommits.clear();
}
```

Listing 3.2: Initialisation of analysis output in `MyAnalysis`

The initialisation of the analysis is performed in the `initializeResults` method of the `MyAnalysis` class. This method prepares the output environment before commit traversal begins by ensuring that the output directory exists and by resetting previously generated result files to guarantee reproducibility across multiple executions.

During initialisation, two output files are created: `commit_analysis_log.txt` and `commit_messages.txt`. The former stores a compact, structured summary of the analysis results for each commit, including commit identifiers and classification information. The latter stores the raw commit messages associated with each analysed commit. Separating these outputs allows structured data to be processed automatically while preserving commit messages for later manual inspection or qualitative analysis.

To prevent duplicate output, the `MyAnalysis` class maintains a set of commit identifiers that have already been processed. Before writing results for a commit, the analysis checks whether the corresponding identifier has already been recorded.

3.3.2 Classification

The classification of variability-related changes is implemented in the `classifyDiff` method of the `MyAnalysis` class. This method operates on the variability-aware diff computed by DiffDetective and iterates over the artifact nodes contained in the diff structure.

```
private EditClass classifyDiff(VariationDiff<?> diff) {
    for (DiffNode<?> node : diff.computeAllNodesThat(n ->
        n.isArtifact())) {
        EditClass editClass = ProposedEditClasses.Instance.match(node);
        if (ProposedEditClasses.Refactoring.equals(editClass)
            || ProposedEditClasses.Reconfiguration.equals(editClass)) {
            return editClass;
        }
    }
    return null;
}
```

Listing 3.3: Classification of variability-aware edits in `MyAnalysis`

For each artifact node, the method applies DiffDetective's proposed edit-class matching mechanism to assign a semantic edit class. The implementation prioritises higher-level edit classes such as refactorings and reconfigurations. If such a classification is detected, the method terminates early and returns the corresponding edit class. If no prioritised edit class is found, the method returns `null`, indicating that no relevant edit class was detected by this heuristic.

In addition to structural change classification, MyAnalysis analyses commit messages to identify commits that are likely related to bug fixing. This logic is implemented using a lightweight keyword-based heuristic.

```
private boolean isBugRelated(String message) {
    String lower = message.toLowerCase();
    return lower.contains("bug")
        || lower.contains("fix")
        || lower.contains("error")
        || lower.contains("issue")
        || lower.contains("defect");
}
```

Listing 3.4: Heuristic for detecting bug-related commits

A commit is classified as bug-related if its message contains commonly used terms associated with bug fixes, such as *bug*, *fix*, *error*, *issue*, or *defect*. While this heuristic does not guarantee perfect accuracy, it enables efficient labelling of commits and supports later grouping and interpretation of results.

3.4 Topic Analysis

The topic analysis is implemented as a lightweight post-processing step that operates on the output produced by MyAnalysis. Its purpose is to structure the commit-level results into higher-level themes by analysing the vocabulary used in commit messages for selected categories of commits.

3.4.1 Input Data

The topic analysis consumes the files `commit_analysis_log.txt` and `commit_messages.txt`. The first file contains one structured summary line per commit, including the commit identifier and assigned edit class. The second file contains the raw commit messages.

Commits classified as *Refactoring* or *Reconfiguration* are selected by parsing `commit_analysis_log.txt`. Only the commit messages corresponding to these identifiers are included in the subsequent analysis.

```
refac_types = {"Refactoring", "Reconfiguration"}
if ctype in refac_types:
    interesting_ids.add(cid)
```

Listing 3.5: Selecting refactoring and reconfiguration commits from the analysis log

3.4.2 Keyword Frequency Analysis

To approximate topics, a keyword frequency analysis is performed over the selected commit messages. Messages are tokenised into lowercase words using a regular-expression-based tokenizer. Common stopwords and noise terms are removed, and very short tokens are excluded. The remaining words are counted to produce a ranked list of frequently occurring keywords.

```
text = re.sub(r'[^a-zA-Z\s]', '', text).lower()
words = [w for w in text.split() if len(w) > 2]
counter = Counter(words)
most_common = counter.most_common(10)
```

Listing 3.6: Keyword frequency analysis over selected commit messages

3.4.3 Output and Limitations

The results of the topic analysis are written to a human-readable report file. The report includes the number of analysed commits, the most frequent keywords, and the commit messages used for the analysis. This supports both quantitative inspection and qualitative validation.

The topic analysis is intentionally lightweight and does not claim to perform semantic topic modelling. Instead, it provides a transparent, frequency-based approximation of recurring themes. The results depend on the quality and consistency of commit messages and may reflect repository-specific terminology. Despite these limitations, the approach is efficient and suitable for supporting systematic interpretation of large commit histories.

3.5 Evaluation

3.5.1 Setup

3.5.2 Results

3.5.3 Threats to Validity

[1]

Chapter 4 Discussion

Chapter 5 Conclusion

List of Figures

2.1 Classification of edits to configurable software proposed by Bittner et al. (2022). . 6

List of Tables

Bibliography

[1] Claes Wohlin et al. *Experimentation in Software Engineering, Second Edition*. Springer, 2024. DOI: 10.1007/978-3-662-69306-3.